

基于微内核的自演进 RAN 原型

系统详细介绍与用户手册 (V2)

项目名称: 基于微内核的自演进 RAN 原型 (Microkernel-based Self-Evolving RAN Prototype)

当前版本: Alpha 1.0 — 模块1-6 全阶段集成版, 含系统首页与频段配置

文档用途: 科研项目立项汇报 / 系统申报 / 团队协作参考

更新日期: 2026-05-22

上一版本: v0.5.0 (V1 用户手册见 [系统介绍与用户手册.md](#))

目录

- 第一部分: 项目概述
 - 1.1 研究背景与问题动机
 - 1.2 项目定位与核心创新
 - 1.3 与传统 AI-RAN 范式的区别
 - 1.4 全阶段验证目标与达成情况
- 第二部分: 系统架构设计
 - 2.1 总体架构概览
 - 2.2 核心设计原则
 - 2.3 分层架构详解
 - 2.4 核心数据流与业务闭环
- 第三部分: 项目完整目录结构与文件说明
 - 3.1 项目根目录
 - 3.2 AI 智能体编排层 ([uran/ai/](#))
 - 3.3 数字孪生验证层 ([uran/twin/](#))
 - 3.4 通信微内核运行时 ([uran/runtime/](#))
 - 3.5 通信模块库 ([uran/modules/](#))
 - 3.6 SDR 半实物适配 ([uran/sdr/](#))
 - 3.7 自演进闭环 ([uran/evolution/](#))
 - 3.8 仿真核心层 ([uran/sim/](#))
 - 3.9 Web 可视化层 ([uran/dashboard/](#))
 - 3.10 端到端集成层 ([uran/demo/](#))
 - 3.11 公共工具层 ([uran/common/](#) & [uran/utils/](#))

- 3.12 测试目录 (tests/)
 - 3.13 配置与文档目录
- 第四部分：模块1-6 已实现功能清单
- 第五部分：用户手册
 - 5.1 环境准备与安装
 - 5.2 快速启动指南
 - 5.3 界面操作指南
 - 5.4 各页面功能介绍
 - 5.5 KPI 指标解读
 - 5.6 数据导出指南
 - 5.7 编程接口使用指南
 - 5.8 运行测试
 - 5.9 常见问题 FAQ
- 5.10 系统部署指南
- 第六部分：多阶段演进路线图
- 第七部分：项目现状与产出清单
- 附录

第一部分：项目概述

1.1 研究背景与问题动机

1.1.1 通信系统的"固化之困"

当今 4G/5G 移动通信系统虽然功能强大，但它们有一个共同特点：**通信协议栈由人类工程师一次性设计并标准化完成，部署后难以做出结构性调整**。这带来了几个深层问题：

- 场景适应性不足**：一套协议栈需要覆盖从城市宏蜂窝到农村广覆盖、从高速铁路到室内热点等千差万别的场景。在极端条件下（如灾害应急通信、卫星通信、无人集群），既定参数往往不是最优的。
- 升级迭代周期漫长**：从 3GPP 标准化到设备商用通常需要数年时间。当新场景、新需求出现时，系统无法快速响应。
- "一刀切"的设计**：现有系统对所有用户、所有场景使用固定的调制编码表、固定的 HARQ 策略、固定的导频结构。而在动态环境中，这些"最优配置"可能瞬间变为"最差配置"。

1.1.2 AI 进入 RAN 的现状与局限

当前学术界和产业界（包括 3GPP Rel-18/19、O-RAN Alliance）正在探索 AI-RAN，主要做法是：

传统 RAN 架构 + AI 局部优化

例如用 AI 增强信道估计、波束管理、资源调度、流量预测和节能控制。这些工作有价值，但本质上仍是 "人工设计通信系统 + AI 局部优化" 的范式——通信系统的基本架构、协议栈结构、模块边界仍然是人工预设的。

1.1.3 本项目试图回答的问题

能否进一步推进到"AI 按需生成通信能力"的新阶段？

即：系统初始不预置完整、固定的空口和协议栈，而是保留一个**可信、稳定、低复杂度、可回退的最小通信内核**，在此基础上由 AI 根据任务需求、信道状态、频谱环境、终端能力、算力资源和 QoS 约束，动态选择、组合、配置甚至受限生成通信功能模块。

1.2 项目定位与核心创新

1.2.1 系统定位

基于微内核的自演进 RAN 原型 (Microkernel-based Self-Evolving RAN Prototype) 定位为面向未来 AI-Native RAN 的科研原型平台，是对当前 AI-RAN 范式的一次纵深推进。

核心思想可概括为：

维度	传统 RAN	本系统
通信能力	人一次性固化设计	AI 按需动态生成
协议栈	固定分层，预先实现	可插拔模块，动态组装
安全兜底	依赖完整协议栈	微内核作为保底生命线

1.2.2 核心创新点

- 1. **通信微内核**：借鉴 Linux 微内核思想，保留最小、可信、稳定的通信能力作为系统底座（低速可靠控制信道 + 基础同步 + 最小信令 + 回退机制），而非依赖完整协议栈。

2. **AI 受约束生成**: AI 不是在无边界空间内自由生成, 而是在模块仓库、安全规则引擎和数字孪生验证的多重约束下进行"受约束创作"。
 3. **全链路闭环验证**: 所有 AI 生成的候选方案必须先进入数字孪生沙箱完成完整验证 (接口检查 + 约束验证 + 链路仿真 + 故障注入 + 安全回退测试), 只有综合评分合格的方案才能签名准入。
 4. **安全自愈回退**: Runtime 中实时监控模块运行状态, 一旦异常 (BLER 超标、时延过限、模块崩溃), 自动卸载异常模块并切回微内核保底通道。
 5. **持续演进闭环**: 运行数据反哺数字孪生模型校准 → 模块性能迭代 → AI 策略再生成 → 模块库版本更新, 形成自学习、自优化、自演进闭环。
-

1.3 与传统 AI-RAN 范式的区别

传统 AI-RAN:
人工设计完整协议栈 → 部署上线 → AI 在固定框架内局部优化
(如 AI 增强信道估计、AI 辅助调度)

本系统:
最小可信微内核 → AI 按需生成/组合通信模块 → 数字孪生验证 → Runtime 动态部署 → 异常回退微内核
(通信能力本身由 AI 生成, 不止是优化)

对比维度	传统 AI-RAN	本系统
通信能力来源	人工预先设计	AI 动态生成 + 组合
协议栈形态	固化的分层实现	可插拔模块生态
AI 的角色	优化器 (参数调优)	生成器 (模块配置/组合/受限代码)
安全机制	依赖完整协议栈	微内核兜底 + 安全回退
适应周期	月/年级标准化	秒/分级在线调整
适用场景	常规蜂窝通信	应急/卫星/专用/无人集群等极端场景

1.4 全阶段验证目标与达成情况

本系统已按五阶段路线图完成全部开发, 每个阶段均通过 104 项自动化测试验证。

阶段	名称	核心验证目标	状态
模块1	链路自适应仿真与KPI可视化	最小通信内核可行性、信道动态模拟、自适应决策档位化、Web可视化	✅ 已达成
模块2	AI意图解析与方案生成	自然语言需求理解、多智能体协同编排、差异化方案自动生成	✅ 已达成
模块3/4	数字孪生验证 + 微内核Runtime	接口/约束验证、链路仿真、故障注入、模块热加载、安全回退	✅ 已达成
模块5	Mock SDR半实物演示	IQ波形生成、AWGN信道模拟、BER/EVM测量、频谱分析	✅ 已达成
模块6	自演进闭环优化	经验持久化、策略记忆、启发式优化、多轮迭代演进	✅ 已达成

第二部分：系统架构设计

2.1 总体架构概览

```text





## 2.2 核心设计原则

| 原则       | 说明                          | 来源          |
|----------|-----------------------------|-------------|
| 微内核极简    | 内核只保留生命线能力，不承载业务逻辑          | Linux 微内核思想 |
| ABI 稳定   | 模块接口主版本稳定，允许扩展不破坏兼容         | 操作系统设计      |
| AI 受约束生成 | AI 在模块仓库、安全规则、数字孪生三重约束下创作   | Linux eBPF  |
| 先验证后部署   | 所有方案必须经过数字孪生验证、评分才能上线       | 安全关键系统      |
| 控制面保底    | 独立低速可靠控制信道始终存在，作为系统生命线和回退通道 | 电信级可靠性      |
| 可回退优先    | 任何新能力失败时，系统自动卸载异常模块并切回微内核   | 故障安全设计      |
| 数据闭环     | 运行数据反哺孪生模型和 AI 编排器，实现持续演进   | 持续学习        |

## 第三部分：项目完整目录结构与文件说明

以下列出项目全部源码、配置和文档文件，按模块分组说明。文件总数：75+ 个 Python 源文件、10 个测试文件、4 个 YAML 场景配置、8+ 份技术文档

### 3.1 项目根目录

|                                        |                                                                                                        |
|----------------------------------------|--------------------------------------------------------------------------------------------------------|
| e:\AI RAN Demo\                        |                                                                                                        |
| ├─ LOGO.jpg                            | ← 系统图标                                                                                                 |
| ├─ README.md                           | ← （可添加）项目简介                                                                                            |
| ├─ requirements.txt                    | ← Python 依赖清单                                                                                          |
| ├─ 首页设计.md                             | ← 系统首页 UI 设计文档                                                                                         |
| ├─ RAN-AI Phase II-V 科研演示系统完整详细设计文档.md | ← 模块2-6 完整设计文档                                                                                         |
| ├─ 设计思路.txt                            | ← 原始设计理念、架构原则                                                                                          |
| ├─ 初步设计.txt                            | ← 架构设计/概要设计/框架设计/模块                                                                                    |
| 划分                                     |                                                                                                        |
| ├─ 详细设计.txt                            | ← 面向 AI IDE 的实现级设计说明                                                                                   |
| ├─ uran/                               | ←  主源码包（详见下方各子目录）    |
| ├─ tests/                              | ←  测试目录（10 个测试文件，104 |
| 个用例）                                   |                                                                                                        |
| ├─ configs/                            | ←  配置目录              |
| │   ├─ research_scenarios/             | ← 科研场景 YAML 配置                                                                                         |
| ├─ docs/                               | ←  文档目录              |
| ├─ outputs/                            | ←  输出目录（仿真结果、导出报     |
| 告）                                     |                                                                                                        |
| └─ .pytest_cache/                      | ← pytest 缓存（自动生成）                                                                                      |

## 3.2 AI 智能体编排层 ( uran/ai/ )

**功能：**接收自然语言任务需求，通过多智能体协同生成通信方案。 **对应模块：**模块2

| 文件                                     | 说明                                                                                                              |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>               | 模块导出接口，导出 AIOrchestrator、IntentSpec、PlanSpec 等                                                                  |
| <code>intent_schema.py</code>          | <b>意图与环境数据结构</b> — 定义 <code>IntentSpec</code> （用户需求规格）和 <code>EnvironmentSpec</code> （环境参数规格），将自然语言需求结构化为可计算的数据 |
| <code>plan_schema.py</code>            | <b>通信方案数据结构</b> — 定义 <code>PlanSpec</code> ，包含调制方式、编码方案、编码率、MCS、HARQ配置、导频密度、发射功率、调度策略、功控策略等全部通信参数               |
| <code>agent_base.py</code>             | <b>智能体基类</b> — 所有 AI Agent 的抽象基类，定义 <code>process()</code> 统一接口和通用日志追踪能力                                        |
| <code>intent_agent.py</code>           | <b>意图解析智能体</b> — 将中文自然语言需求（如“在高铁场景下需要高可靠性通信，BLER<1%”）解析为结构化的 <code>IntentSpec</code> （场景类型、优先级、性能目标等）           |
| <code>environment_agent.py</code>      | <b>环境分析智能体</b> — 分析环境参数（SNR、移动速度、干扰水平、节点数量等），生成 <code>EnvironmentSpec</code> ，支持外部传入环境参数                        |
| <code>module_selection_agent.py</code> | <b>模块选择智能体</b> — 根据意图和环境，从通信模块库中选择合适的调制、编码、HARQ、调度等模块组合                                                         |
| <code>parameter_gen_agent.py</code>    | <b>参数生成智能体</b> — 为选中的模块填充具体参数值（MCS等级、编码率、重传次数、发射功率等）                                                            |
| <code>critic_agent.py</code>           | <b>方案审查智能体</b> — 对生成方案进行审查，标记潜在问题并给出改进建议                                                                        |
| <code>llm_adapter.py</code>            | <b>LLM 适配器（预留）</b> — 为未来接入大语言模型（GPT/Claude/本地模型）预留接口，当前使用规则基线                                                   |
| <code>orchestrator.py</code>           | <b>AI 编排器</b> — 多智能体协同总入口，协调五个 Agent 的顺序执行，生成多套候选方案                                                             |

### 3.3 数字孪生验证层 ( `uran/twin/` )

**功能：**在虚拟沙箱中对候选通信方案进行五步验证，输出评分报告。**对应模块：**模块3

| 文件                              | 说明                                                                                                                                         |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>        | 模块导出接口                                                                                                                                     |
| <code>sandbox.py</code>         | <b>数字孪生沙箱</b> — 集成验证主入口，依次执行接口检查→约束验证→链路仿真→故障注入→综合评分，返回 <code>TwinValidationReport</code> ，决策为 ACCEPT/NEED_REVISE/REJECT                   |
| <code>validators.py</code>      | <b>接口与约束验证器</b> — <code>InterfaceValidator</code> 检查方案字段完整性（10个必需字段）， <code>ConstraintValidator</code> 检查参数合法性（调制方式可支持性、编码率范围、MCS范围、功率上限等） |
| <code>link_simulator.py</code>  | <b>链路级仿真器</b> — <code>LinkLevelSimulator</code> 基于有效 SINR 计算 BLER/吞吐量/时延/频谱效率/能耗，考虑编码增益、导频效率、HARQ 增益、调制惩罚、干扰惩罚                             |
| <code>fault_injection.py</code> | <b>故障注入器</b> — <code>FaultInjector</code> 模拟 SNR 骤降、突发干扰、高移动性、功率限制、模块故障五种故障，测试方案的鲁棒性                                                       |
| <code>scoring.py</code>         | <b>综合评分器</b> — <code>TwinScorer</code> 按优先级（可靠性/吞吐/时延/能耗/均衡/抗干扰）加权计算 0-100 分，包含鲁棒性分项                                                       |
| <code>report.py</code>          | <b>验证报告数据结构</b> — <code>TwinValidationReport</code> 包含决策、各检查项通过状态、KPI 值、警告/错误/建议                                                           |

### 3.4 通信微内核运行时 ( `uran/runtime/` )

**功能：**可信底座，负责模块加载、运行监控和安全回退。**对应模块：**模块4

| 文件                            | 说明                                                                                                                                                                                                            |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>      | 模块导出接口，导出 <code>MicrokernelRuntime</code> 、 <code>RuntimeState</code>                                                                                                                                         |
| <code>microkernel.py</code>   | <b>微内核核心</b> — <code>MicrokernelRuntime</code> 提供 <code>load_plan()</code> 加载方案、 <code>step()</code> 单步执行、 <code>run()</code> 批量执行、 <code>inject_failure()</code> 故障注入、 <code>fallback()</code> 安全回退，是系统的执行引擎 |
| <code>state_store.py</code>   | <b>状态存储</b> — <code>RuntimeState</code> 数据结构，保存运行时ID、活跃方案ID、活跃模块列表、状态(RUNNING/FALLBACK)、步数、最近KPI、事件列表、回退原因                                                                                                    |
| <code>events.py</code>        | <b>事件总线</b> — <code>EventBus</code> 记录系统运行事件（ <code>PLAN_LOADED</code> 、 <code>MODULE_FAILURE_INJECTED</code> 、 <code>FALLBACK_ACTIVATED</code> 等），支持时间戳查询                                                    |
| <code>monitor.py</code>       | <b>运行时监控</b> — <code>RuntimeMonitor</code> 收集每一步的 KPI 数据，支持近期 BLER 统计和综合摘要                                                                                                                                    |
| <code>safety_guard.py</code>  | <b>安全守护</b> — <code>SafetyGuard</code> 检测异常（模块健康检查失败、连续高 BLER），判定是否需要触发回退                                                                                                                                     |
| <code>module_loader.py</code> | <b>模块加载映射</b> — <code>map_plan_to_module_ids()</code> 将通信方案转换为可执行模块 ID 列表（如 QPSK→ <code>mod_qpsk</code> 、卷积编码→ <code>code_conv_1_2</code> ）                                                                   |

### 3.5 通信模块库 ( `uran/modules/` )

**功能：**可插拔通信模块集合，由微内核动态加载执行。 **对应模块：**模块4

| 文件                            | 说明                                                                                                                                                                                                                                          |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>      | 模块库导出接口                                                                                                                                                                                                                                     |
| <code>base.py</code>          | <b>模块基类</b> — <code>CommunicationModule</code> 定义统一接口： <code>configure()</code> 配置、 <code>process()</code> 处理、 <code>health_check()</code> 健康检查、 <code>inject_failure()</code> 故障注入、 <code>recover()</code> 恢复、 <code>describe()</code> 自描述 |
| <code>registry.py</code>      | <b>模块注册中心</b> — <code>ModuleRegistry</code> 管理所有模块， <code>build_default_registry()</code> 构建包含 18+ 模块的默认注册表                                                                                                                                 |
| <code>modulation.py</code>    | <b>调制模块</b> — BPSK/QPSK/16QAM/64QAM，每符号 1/2/4/6 比特                                                                                                                                                                                          |
| <code>coding.py</code>        | <b>编码模块</b> — 重复编码(1/2、1/3)、卷积编码(1/2、2/3)、LDPC(2/3、3/4)                                                                                                                                                                                     |
| <code>harq.py</code>          | <b>HARQ 重传模块</b> — 无HARQ、停等重传、增量冗余，含重传增益                                                                                                                                                                                                    |
| <code>pilot.py</code>         | <b>导频密度模块</b> — low/medium/high 三档，影响信道估计精度和频谱效率                                                                                                                                                                                            |
| <code>scheduler.py</code>     | <b>调度器模块</b> — throughput_first/reliability_first/latency_first/balanced/fairness                                                                                                                                                           |
| <code>power_control.py</code> | <b>功控模块</b> — snr_based/fixed/conservative/aggressive/energy_saving                                                                                                                                                                         |
| <code>fallback.py</code>      | <b>安全回退模块</b> — <code>FallbackSafeBPSKModule</code> 系统最后防线：BPSK+1/3 码率+高导频+保守功控                                                                                                                                                             |

### 3.6 SDR 半实物适配 ( uran/sdr/ )

**功能：**无线电信号生成、信道模拟和性能测量。**对应模块：**模块5

| 文件                               | 说明                                                                                                                                                                        |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>         | 模块导出接口                                                                                                                                                                    |
| <code>adapter_base.py</code>     | <b>SDR 适配器抽象基类</b> — 定义统一接口： <code>configure()</code> 、 <code>transmit()</code> 、 <code>measure_spectrum()</code> 、 <code>measure_metrics()</code> 、 <code>close()</code> |
| <code>mock_sdr.py</code>         | <b>模拟 SDR 适配器</b> — <code>MockSDRAdapter</code> 在无硬件环境下模拟无线电收发声、测量 BER/EVM/丢包率                                                                                            |
| <code>waveform.py</code>         | <b>波形生成</b> — <code>generate_iq_symbols()</code> 生成 BPSK/QPSK/16QAM/64QAM<br><code>add_awgn()</code> 按 SNR 添加高斯白噪声                                                        |
| <code>spectrum.py</code>         | <b>频谱分析</b> — <code>estimate_psd()</code> 使用 FFT+汉宁窗估计功率谱密度                                                                                                               |
| <code>gnuradio_adapter.py</code> | <b>GNU Radio 适配器 (预留)</b> — 为未来接入 GNU Radio+USRP 硬件预                                                                                                                      |

### 3.7 自演进闭环 ( `uran/evolution/` )

**功能：**经验持久化、策略推荐和持续优化。 **对应模块：**模块6

| 文件                              | 说明                                                                                                    |
|---------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>        | 模块导出接口                                                                                                |
| <code>experience.py</code>      | <b>经验数据结构</b> — <code>Experience</code> 数据类，记录意图、环境、方案、孪生评分、运行KPI、成功/失败、失败原因、优化建议                     |
| <code>data_lake.py</code>       | <b>数据湖</b> — <code>DataLake</code> 将经验以 JSONL 格式持久化到磁盘，支持追加写入、全量加载和按场景查询                              |
| <code>strategy_memory.py</code> | <b>策略记忆</b> — <code>StrategyMemory</code> 按场景保存历史最优策略，基于综合得分（孪生评分+成功率+吞吐+BLER+时延）排序，保留 Top-K          |
| <code>optimizer.py</code>       | <b>启发式优化器</b> — <code>HeuristicOptimizer</code> 基于 KPI 反馈生成优化建议：BLER 高→增加可靠性，吞吐低→提升效率，时延高→降低时延，全达标→微调 |
| <code>recommender.py</code>     | <b>方案推荐器</b> — <code>SimpleRecommender</code> 基于历史经验和策略记忆，为给定场景推荐最优通信方案                               |
| <code>evolution_loop.py</code>  | <b>演进闭环</b> — <code>EvolutionLoop</code> 编排完整演进流程：AI生成方案→孪生验证→微内核执行→KPI分析→优化建议→经验存储，支持单轮和多轮迭代         |

### 3.8 仿真核心层 ( uran/sim/ )

**功能：** 底层通信仿真引擎，所有算法的执行和数据生成。 **对应模块：** 模块1

| 文件                               | 说明                                                                                                         |
|----------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>         | 仿真模块导出接口                                                                                                   |
| <code>node.py</code>             | <b>节点模型</b> — 定义通信节点 (gNB/UE/Relay/Jammer/Satellite/EdgeServer)、发射功率                                       |
| <code>link.py</code>             | <b>链路模型</b> — 定义通信链路及其自适应参数状态 (MCS、调制、编码率)                                                                 |
| <code>topology.py</code>         | <b>拓扑管理</b> — 管理节点-链路关系、距离计算、拓扑验证                                                                          |
| <code>channel_timeline.py</code> | <b>信道时间线</b> — 支持 7 种时变信道模式 (constant/linear_fading/sinusoidal/random_walk/blockage/jamming) 实时计算 SNR/SINR |
| <code>adaptation.py</code>       | <b>自适应控制器</b> — 4 档决策 (高速→均衡→可靠→保底)，含 BLER 历史                                                              |
| <code>kpi_model.py</code>        | <b>KPI 估算模型</b> — 基于信道质量和通信参数估算 BLER/吞吐量/时延/频谱指标                                                           |
| <code>kpi_recorder.py</code>     | <b>KPI 记录器</b> — 时间序列记录，支持 DataFrame 导出                                                                    |
| <code>experiment.py</code>       | <b>实验运行器</b> — step/reset/run_all 仿真控制流程，统一调度各模块                                                           |
| <code>scenario_loader.py</code>  | <b>场景加载器</b> — 从 YAML 配置构建完整仿真环境 (拓扑+信道+自适应+)                                                              |

### 3.9 Web 可视化层 ( uran/dashboard/ )

**功能：** 基于 Streamlit + Plotly 的 Web 仪表盘，8 个交互页面覆盖系统首页与全阶段。 **对应模块：** 模块1-6

| 文件                                  | 说明                                                                                                                            |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>            | Dashboard 模块导出接口                                                                                                              |
| <code>research_app.py</code>        | <b>主应用入口</b> — 8 页面路由（系统首页 / 模块1 KPI可视化 / 模块2 AI编排 / 模块3 孪生验证 / 模块4 微内核监控 / 模块5 SDR演示 / 模块6 演进优化 / 端到端全流程），含侧边栏导航、频段配置和会话状态管理 |
| <code>research_controller.py</code> | <b>仿真控制器</b> — 封装仿真操作（加载场景/初始化/步进/重置），含频段配置切换                                                                                 |
| <code>topology_view.py</code>       | <b>拓扑视图</b> — Plotly 绘制网络拓扑图，节点颜色编码（gNB蓝/UE绿/Jammer红），链路颜色反映 BLER（绿色好/橙色警告/红色恶化）                                              |
| <code>kpi_charts.py</code>          | <b>KPI 曲线图表</b> — 6 种 Plotly 时间序列曲线：SNR/SINR、BLER、吞吐量、时延、自适应参数（MCS/调制等）、可靠性                                                   |
| <code>timeline_view.py</code>       | <b>事件时间线</b> — 链路自适应事件追踪（MCS 切换/调制变化/功率调整），可视化系统对信道变化的响应                                                                      |
| <code>report_exporter.py</code>     | <b>报告导出</b> — CSV/JSON/HTML/Markdown 多格式导出，含实验摘要生成（KPI Summary + Interpretation）                                              |

### 3.10 端到端集成层 ( `uran/demo/` )

**功能：**一键运行全流程，从自然语言需求到 SDR 示波器。 **对应模块：**模块1-6 集成

| 文件                         | 说明                                                                                                                                                    |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>   | Demo 模块导出接口                                                                                                                                           |
| <code>end_to_end.py</code> | <b>端到端 Demo</b> — <code>run_end_to_end_demo()</code> 一键执行：AI 生成方案 → 数字孪生验证 → 微内核执行 → SDR 演示 → 演进优化推荐 → 经验存储。返回 <code>DemoReport</code> 包含 11 个维度的完整结果 |

### 3.11 公共工具层 ( `uran/common/` & `uran/utils/` )

**功能：**跨模块共享的基础工具和常量。

| 文件                           | 说明                                                                          |
|------------------------------|-----------------------------------------------------------------------------|
| uran/common/__init__.py      | 公共模块导出接口                                                                    |
| uran/common/ids.py           | <b>统一 ID 生成</b> — <code>new_id()</code> 生成唯一标识符（如 plan_xxxx、intent_xxxx）    |
| uran/common/time_utils.py    | <b>时间工具</b> — <code>now_iso()</code> 返回 ISO 格式 UTC 时间戳                      |
| uran/common/serialization.py | <b>序列化工具</b> — <code>to_dict()</code> 将 dataclass 对象递归转换为字典                 |
| uran/common/exceptions.py    | <b>自定义异常</b> — 项目专用异常类定义                                                    |
| uran/common/demo_defaults.py | <b>演示系统默认参数</b> — 编码增益表、导频增益表、调制惩罚表、调制阈值表、调度时延表等常量定义                        |
| uran/common/band_config.py   | <b>频段配置</b> — 频段方案数据类（BandConfig），含 n41/n79 中国移动预置方案，支持自定义频段；频率损耗计算影响全部 KPI |
| uran/utils/__init__.py       | 工具模块导出接口                                                                    |
| uran/utils/file_utils.py     | <b>文件工具</b> — 目录创建、路径处理等辅助函数                                                |

### 3.12 测试目录 ( tests/ )

**功能：** 自动化测试，覆盖全部模块，共 119 个测试用例。

| 文件                          | 测试范围                   | 用例数 |
|-----------------------------|------------------------|-----|
| test_adaptation.py          | 链路自适应决策逻辑              | 4   |
| test_channel_timeline.py    | 信道变化模型                 | 5   |
| test_dashboard_exporter.py  | 实验报告导出                 | 3   |
| test_research_simulation.py | 科研场景仿真运行               | 5   |
| test_ai_orchestrator.py     | AI 编排器与意图解析            | 7   |
| test_digital_twin.py        | 数字孪生沙箱验证               | 10  |
| test_microkernel.py         | 微内核运行时与通信模块            | 30  |
| test_sdr.py                 | SDR 信号处理与模拟            | 15  |
| test_evolution.py           | 自演进闭环优化                | 12  |
| test_end_to_end.py          | 端到端全流程集成               | 8   |
| test_band_config.py         | 频段配置模块（预设方案/自定义/KPI联动） | 15  |
| 合计                          |                        | 119 |

### 3.13 配置与文档目录

#### 场景配置 ( configs/research\_scenarios/ )

| 文件                          | 说明                         |
|-----------------------------|----------------------------|
| p2p_link_adaptation.yaml    | 点对点链路自适应 — 用户远离基站的模拟       |
| multi_node_cell.yaml        | 多用户蜂窝小区 — 3 个用户在不同距离的差异化体验 |
| jamming_self_evolution.yaml | 干扰自演进 — 恶意干扰器突然出现的攻击场景     |
| blockage_recovery.yaml      | 遮挡恢复 — 建筑物遮挡后的信号逐步恢复       |

#### 文档 ( docs/ )

| 文件                  | 说明                          |
|---------------------|-----------------------------|
| 系统介绍与用户手册.md        | V1 版本文档（模块1 演示版）            |
| 概要设计文档.md           | 系统架构、模块划分                   |
| 模块设计文档.md           | 各模块接口、数据模型                  |
| 详细设计文档.md           | 类设计、方法签名                    |
| 测试报告.md             | 模块1 的 17 个测试用例详解            |
| AI_RAN_Demo_测试报告.md | 全阶段 104 个测试用例详解（最新版）        |
| 设计文档/               | 子目录，含 PDF 版设计文档及 模块2-6 设计文档 |

---

## 第四部分：模块1–6 已实现功能清单

---

| 阶段   | 功能模块     | 关键能力                                            | 测试覆盖 |
|------|----------|-------------------------------------------------|------|
| 模块 1 | 仿真核心层    | 节点/链路/拓扑管理、7种信道模式、4档自适应决策、KPI估算                 | 14 项 |
| 模块 1 | Web 可视化层 | Streamlit 仪表盘、Plotly 拓扑图/KPI曲线/事件时间线、多格式导出      | 3 项  |
| 模块 2 | AI 编排层   | 5 Agent 协同、自然语言意图解析、多候选方案生成                     | 7 项  |
| 模块 3 | 数字孪生验证层  | 接口/约束/链路/故障/评分五步验证、ACCEPT/REJECT/NEED_REVISE 决策 | 10 项 |
| 模块 4 | 微内核运行时   | 模块加载/执行/监控、事件总线、安全守护、故障注入→回退                    | 30 项 |
| 模块 4 | 通信模块库    | 18+ 可插拔模块（调制×4、编码×6、HARQ×3、导频×3、调度×5、功控×5、回退×1） | —    |
| 模块 5 | SDR 半实物  | IQ 波形生成（4种调制）、AWGN 信道模拟、FFT 频谱分析、BER/EVM 测量     | 15 项 |
| 模块 6 | 自演进闭环    | JSONL 经验持久化、策略记忆 Top-K、启发式优化（4种策略）、多轮迭代         | 12 项 |
| 集成   | 端到端流程    | 一键运行全流程、11 维度 DemoReport、确定性可复现                 | 8 项  |

## 第五部分：用户手册

### 5.1 环境准备与安装

### 5.1.1 系统要求

| 项目     | 要求                              |
|--------|---------------------------------|
| 操作系统   | Windows 10+ / macOS 11+ / Linux |
| Python | 3.10+ (开发使用 3.13.12)            |
| 内存     | ≥ 4 GB                          |
| 磁盘     | ≥ 500 MB                        |

### 5.1.2 安装步骤

```
1. 进入项目目录
cd "e:\AI RAN Demo"

2. 创建虚拟环境（推荐）
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # macOS/Linux

3. 安装依赖
pip install -r requirements.txt
```

### 5.1.3 主要依赖

| 库         | 用途                 |
|-----------|--------------------|
| streamlit | Web 仪表盘框架          |
| plotly    | 交互式图表（拓扑图/KPI曲线）   |
| pandas    | 数据处理与 DataFrame 导出 |
| numpy     | 数值计算（噪声/随机数/FFT）   |
| pytest    | 自动化测试框架            |
| pyyaml    | YAML 场景配置解析        |

## 5.2 快速启动指南

### 5.2.1 启动 Web 仪表盘

```
cd "e:\AI RAN Demo"
python -m streamlit run uran/dashboard/research_app.py --server.headless true
```

启动成功后，在浏览器中打开 <http://localhost:8501>。

### 5.2.2 启动后你将看到

- 1. **左侧边栏** — 系统品牌标识、简介、8 个页面导航（含系统首页）、频段配置选择器
- 2. **中央主区域** — 首页介绍或各模块的 KPI 曲线、拓扑图、方案详情等

### 5.2.3 频段配置

系统支持选择不同频段方案，频段选择直接影响所有模块的 KPI 计算（带宽决定吞吐量上限，载波频率决定路径损耗）：

| 方案                | 说明       | 中心频率         | 带宽        |
|-------------------|----------|--------------|-----------|
| n41 2.6GHz 100MHz | 中国移动主力频段 | 2600 MHz     | 100 MHz   |
| n79 4.9GHz 100MHz | 高速热点覆盖   | 4900 MHz     | 100 MHz   |
| n41 2.6GHz 50MHz  | 中等带宽     | 2600 MHz     | 50 MHz    |
| 仿真默认 500kHz       | 小带宽快速仿真  | 2600 MHz     | 0.5 MHz   |
| 自定义               | 自由调节     | 700–6000 MHz | 1–200 MHz |

频段配置位于左侧边栏下方，切换后所有模块自动使用新参数。

## 5.3 界面操作指南

### 5.3.1 侧边栏导航

Dashboard 提供 8 个页面，通过左侧边栏的 Radio 按钮切换。侧边栏顶部显示系统品牌标识和简介，下方为页面导航，最底部为频段配置：

|                            |                                    |
|----------------------------|------------------------------------|
| [LOGO]                     | 基于微内核的自演进 RAN 原型                   |
|                            | Microkernel-based Self-Evolving... |
|                            | 基于通信微内核的 AI 自设计...                 |
| 导航                         |                                    |
| ○ 系统首页                     |                                    |
| ○ 模块1: 链路自适应仿真与KPI可视化      |                                    |
| ○ 模块2: AI意图解析与方案生成         |                                    |
| ○ 模块3: 数字孪生方案验证            |                                    |
| ○ 模块4: 微内核Runtime运行监控      |                                    |
| ○ 模块5: Mock SDR半实物演示       |                                    |
| ○ 模块6: 自演进闭环优化             |                                    |
| ○ 端到端: 全流程一键演示             |                                    |
| 频段配置 [n41 2.6GHz 100MHz ▼] |                                    |
| ▶ 自定义                      |                                    |
| n41                        | 2600MHz 100MHz Δ+0.0dB             |

**系统首页：**展示系统定位、核心关键词和能力，引导用户通过左侧导航进入各功能模块。

5.3.2 模块1 页面操作流程

Page: 模块1: 链路自适应仿真与KPI可视化

模块1 的实验控制已移至页面顶部，以紧凑的一行控制栏呈现：

[场景选择 ▼][初始化][重置][单步][全部][□自动][间隔滑块]

— 分割线 —

实验进度 / 指标 / 拓扑图 / KPI曲线 / 时间线 / 导出...

**推荐流程（用于汇报演示）：**

1. 在页面顶部的「场景」下拉框选择场景 → "点对点链路自适应"

2. 点击「初始化」→ 加载场景配置，创建拓扑和信道模型

3. 点击「全部」→ 一次性运行全部仿真步骤（约1-2秒完成）

4. 观察结果：

- SNR 曲线如何下降

- MCS 如何自动降档

- BLER 如何波动

- 事件时间线追踪每次自适应决策

5. 点击导出按钮 → 生成 CSV/JSON/HTML/Markdown 实验报告

控制按钮说明：

- **初始化**：加载场景配置，初始化仿真环境
- **重置**：重置仿真回到第0步
- **单步**：单步推进（适合教学观察）
- **全部**：一次性跑完全程
- **自动**：自动循环播放（适合展厅无人值守）
- **间隔滑块**：调节自动播放的步进间隔

5.3.3 模块2 页面操作流程

Page: 模块2: AI意图解析与方案生成

1. 在文本框中输入自然语言需求，例如：  
"在SNR=10dB的城区场景下要求BLER<5%，吞吐量>500kbps"  
"高铁场景下需要极高可靠性，BLER<0.1%，弱覆盖"  
"在城市宏基站场景下需要高吞吐低时延，高清视频"
2. 点击 Generate Plan → AI 编排器开始工作
3. 观察结果：
  - 意图解析结果：场景类型、优先级、性能目标
  - 环境分析结果：SNR、移动速度、干扰水平
  - 候选方案：每个方案的调制、编码、MCS 等完整参数
  - 编排追踪链：5 个 Agent 的执行顺序和输出

**多方案对比**：系统会生成 2-3 套候选方案，供数字孪生沙箱进一步验证和比较。

5.3.4 模块3 页面操作流程

Page: 模块3: 数字孪生方案验证

1. 输入自然语言需求（或使用 模块2 生成的结果）
2. 点击 Validate in Digital Twin
3. 观察验证报告：
  - 接口检查：方案是否包含所有必需字段
  - 约束验证：参数是否在合法范围内
  - 链路仿真：BLER/吞吐/时延/频谱效率/能耗
  - 故障注入：SNR骤降和突发干扰下的鲁棒性
  - 综合评分：0-100 分，加权计算
  - 最终决策：ACCEPT / NEED\_REVISE / REJECT

1. 点击 Load Plan → 将验证通过的方案加载到微内核
2. 点击 Run Runtime → 执行通信模块
3. 观察监控面板:
  - Runtime 状态: INIT → RUNNING → FALLBACK
  - KPI 曲线: 每一步的 BLER/吞吐/时延
  - 活跃模块列表: 当前加载的模块清单
  - 事件日志: PLAN\_LOADED、FALLBACK\_ACTIVATED 等
4. 测试安全回退:
  - 点击 Inject Failure → 注入模块故障
  - 观察系统自动切换到 BPSK 保底模式
  - 状态变为 FALLBACK, 事件日志记录回退原因

### 5.3.5 模块5 页面操作流程

1. 配置 SDR 参数: 调制方式、SNR
2. 点击 Configure SDR → 配置模拟无线电
3. 观察结果:
  - IQ 波形显示: 复数信号在星座图上的分布
  - 频谱图: FFT 功率谱密度估计
  - BER 测量: 误比特率 (声明 SNR 下的理论估计)
  - EVM 百分比: 误差向量幅度
  - 丢包率: 模拟信道带来的包丢失
4. 切换不同调制方式 (BPSK/QPSK/16QAM/64QAM) 对比性能

### 5.3.6 模块6 页面操作流程

1. 点击 Run Evolution Iteration → 运行一轮演进
2. 系统自动完成：AI 生成方案 → 数字孪生验证 → Runtime 执行 → KPI分析 → 优化建议
3. 观察结果：
  - 当前方案详情
  - 运行时 KPI 摘要（平均 BLER/吞吐/时延）
  - 优化建议：基于四个方面（可靠性/吞吐/时延/整体）
  - 历史经验统计：累计运行次数、成功/失败比
4. 点击 Run Multiple Iterations → 观察多轮迭代后策略的逐步改善

### 5.3.7 端到端页面操作流程

#### Page: 端到端：全流程一键演示

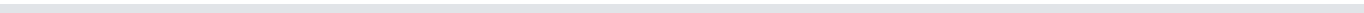
1. 输入自然语言需求或选择预设示例
2. 点击 Run End-to-End → 一键执行完整流程
3. 系统自动完成以下全部步骤：
  - Step 1: AI 意图解析与方案生成
  - Step 2: 数字孪生沙箱验证（5项检查+评分）
  - Step 3: 微内核加载与 Runtime 执行
  - Step 4: Mock SDR 无线信号收发测量
  - Step 5: 自演进优化建议生成
  - Step 6: 运行经验持久化存储
4. 查看 11 个维度的完整 DemoReport：
  - 原始意图 / 解析后的意图 / 环境参数
  - 候选方案列表 / 孪生验证报告列表
  - 最优方案 / 最优方案验证报告
  - Runtime 执行摘要 / SDR 测量摘要
  - 演进优化建议 / 编排追踪链

---

## 5.4 各页面功能介绍

---

| 页面           | 功能定位                  | 适用场景             |
|--------------|-----------------------|------------------|
| 系统首页         | 系统定位与核心能力介绍           | 首次进入、整体了解        |
| 模块1: 链路自适应仿真 | 观察信道变化时系统如何自动调整通信参数   | 基础科研演示、教学展示      |
| 模块2: AI意图解析  | 演示 AI 如何理解自然语言并生成通信方案 | AI-RAN 理念验证、方案对比 |
| 模块3: 数字孪生验证  | 展示方案在虚拟环境中的五步验证流程     | 安全关键论证、方案审核      |
| 模块4: 微内核监控   | 演示模块加载/运行/故障回退的完整生命周期 | 可靠性论证、异常处理演示     |
| 模块5: SDR 演示  | 展示无线信号层面的收发和测量        | 硬件验证预备、信号分析      |
| 模块6: 演进优化    | 展示系统如何从经验中学习和持续改进     | 自适应论证、长期演进       |
| 端到端          | 一键展示从需求到优化的完整闭环       | 立项汇报、高层演示        |



## 5.5 KPI 指标解读



| 指标                  | 全称                                      | 通俗解释                 | 好的值        |
|---------------------|-----------------------------------------|----------------------|------------|
| SNR                 | Signal-to-Noise Ratio                   | 信号"清晰度", 越大信号越清晰     | > 10dB     |
| SINR                | Signal-to-Interference-plus-Noise Ratio | 考虑干扰后的信号清晰度          | > 8dB      |
| BLER                | Block Error Rate                        | 数据传错的概率, 越小越好        | < 5%       |
| BER                 | Bit Error Rate                          | 比特传错的概率, 比BLER更细粒度   | < 1%       |
| Throughput          | Throughput (kbps)                       | 每秒成功传输的数据量           | > 500kbps  |
| Latency             | Latency (ms)                            | 数据从发到收的时间            | < 50ms     |
| Spectral Efficiency | Spectral Efficiency (bps/Hz)            | 单位频谱带宽上的吞吐量          | > 1 bps/Hz |
| MCS                 | Modulation and Coding Scheme            | "通信档位", 值越大速度越快但越易出错 | 0-28       |
| EVM                 | Error Vector Magnitude (%)              | 发射信号质量, 越小越好         | < 10%      |
| Packet Loss         | Packet Loss Rate                        | 网络丢包比例               | < 1%       |

## 5.6 数据导出指南

### 导出方式

Dashboard 的 模块1 页面提供完整的数据导出功能:

| 格式               | 用途            | 适合                 |
|------------------|---------------|--------------------|
| CSV              | 结构化数据表格       | Excel/Python 进一步分析 |
| JSON             | 结构化数据 (嵌套)    | 程序读取、API 数据交换      |
| HTML Figures     | 交互式 Plotly 图表 | 论文补充材料、网页展示        |
| Markdown Summary | 人类可读实验报告      | 项目汇报、团队分享          |

### 导出文件位置

所有导出文件保存在 `outputs/` 目录下：

```
outputs/
├─ figures/ ← HTML 交互图表
├─ experiment_logs/ ← CSV/JSON/Markdown 报告
└─ evolution/ ← 演进经验日志 (experience_log.jsonl)
```

## 5.7 编程接口使用指南

### 5.7.1 端到端 Demo（一键调用）

```
from uran.demo.end_to_end import run_end_to_end_demo
from uran.common.band_config import BAND_PRESETS

report = run_end_to_end_demo(
 "在 SNR=10dB 的城区场景下，要求 BLER<5%，吞吐量>500kbps",
 seed=42,
 band_config=BAND_PRESETS["n41_100mhz"],
)

查看结果
print(report.selected_plan) # 最优通信方案
print(report.runtime_summary) # Runtime 执行摘要
print(report.sdr_summary) # SDR 测量结果
print(report.evolution_recommendation) # 优化建议
```

### 5.7.2 AI 编排器（模块2）

```
from uran.ai.orchestrator import AIOrchestrator

orchestrator = AIOrchestrator(seed=42)
result = orchestrator.generate_plan(
 "在高铁场景下需要高可靠性通信，BLER<1%"
)

intent = result["intent"] # 解析后的意图
env = result["environment"] # 环境参数
plans = result["candidate_plans"] # 候选方案列表
trace = result["trace"] # Agent 执行追踪
```

### 5.7.3 数字孪生验证（模块3）

```

from uran.twin.sandbox import DigitalTwinSandbox
from uran.ai.intent_schema import IntentSpec, EnvironmentSpec

sandbox = DigitalTwinSandbox()
intent = IntentSpec(...)
env = EnvironmentSpec(...)
report = sandbox.validate(plan, intent, env)

print(report.decision) # ACCEPT / NEED_REVISE / REJECT
print(report.final_score) # 0-100
print(report.suggestions) # 改进建议

```

## 5.7.4 微内核运行时（模块4）

```

from uran.runtime.microkernel import MicrokernelRuntime
from uran.modules.registry import build_default_registry

registry = build_default_registry()
runtime = MicrokernelRuntime(registry=registry, seed=42)
runtime.load_plan(plan)
runtime.run(steps=50)

summary = runtime.monitor.summary()
print(summary["avg_bler"])
print(summary["avg_throughput_kbps"])

```

## 5.7.5 演进闭环（模块6）

```

from uran.evolution.evolution_loop import EvolutionLoop

loop = EvolutionLoop(seed=42)
result = loop.run_once("在SNR=10dB的城区场景下要求BLER<5%")

多轮迭代
results = loop.run_iterations("需求描述", n=3)
for r in results:
 print(r["iteration"], r["recommendation"]["changes"])

```

# 5.8 运行测试

## 运行全部测试

```
cd "e:\AI RAN Demo"
python -m pytest tests/ -v
```

## 按模块运行

```
模块1: 链路自适应
python -m pytest tests/test_adaptation.py tests/test_channel_timeline.py -v

模块2: AI编排
python -m pytest tests/test_ai_orchestrator.py -v

模块3/4: 数字孪生 + 微内核
python -m pytest tests/test_digital_twin.py tests/test_microkernel.py -v

模块5: SDR
python -m pytest tests/test_sdr.py -v

模块6: 演进
python -m pytest tests/test_evolution.py -v

集成测试
python -m pytest tests/test_end_to_end.py -v
```

## 测试统计

| 指标   | 数值    |
|------|-------|
| 测试文件 | 10    |
| 测试用例 | 104   |
| 通过率  | 100%  |
| 执行耗时 | < 2 秒 |

## 5.9 常见问题 FAQ

Q1: 启动 Streamlit 时报 `streamlit: command not found` ?

A: 使用 `python -m streamlit run uran/dashboard/research_app.py` 。

Q2: 页面加载后没有数据?

A: 需先点击对应页面的操作按钮。模块1 需先在上方控制栏点击「初始化」再点击「全部」；模块2-6 需点击「生成方案」/「启动演进闭环」等。

Q3: 如何更换端口？

A: 添加 `--server.port 8080` 参数。

Q4: 能否在无 GUI 的服务器上运行？

A: 可以。使用 `--server.headless true` 参数，通过浏览器远程访问。

Q5: 仿真结果是否基于真实物理模型？

A: 使用简化但趋势合理的模型。目标是演示全流程逻辑和可视化效果，而非严格复现 3GPP 物理层。

Q6: 模块2 的 AI 是否接入了大语言模型？

A: 当前使用规则基线（确定性决策），确保可复现性。`llm_adapter.py` 预留了 LLM 接入接口。

Q7: 端到端流程的执行时间是多久？

A: 纯计算时间 < 2 秒。包含 50 步 Runtime 执行 + SDR 模拟 + 演进分析。

Q8: 如何自定义通信场景？

A: 在 `configs/research_scenarios/` 下创建新的 YAML 配置文件，参照已有场景格式。然后在 Dashboard 的 SCENARIO\_FILES 字典中添加映射。

## 5.10 系统部署指南

### 5.10.1 部署方式概览

本系统为纯 Python 项目，无数据库依赖，无外部服务依赖，部署极为简单。

| 部署方式   | 适用场景              | 难度   |
|--------|-------------------|------|
| 本地开发运行 | 个人开发、调试           | ☆    |
| 局域网共享  | 团队内部演示            | ☆☆   |
| 云服务器部署 | 远程访问、长期运行         | ☆☆☆  |
| 打包分发   | 交付给无 Python 环境的用户 | ☆☆☆☆ |

## 5.10.2 本地开发运行

```
cd "项目目录"
pip install -r requirements.txt
python -m streamlit run uran/dashboard/research_app.py
```

浏览器打开 `http://localhost:8501` 。

## 5.10.3 局域网共享

在同一局域网的其他设备上，通过本机 IP 地址访问：

```
python -m streamlit run uran/dashboard/research_app.py --server.headless true
```

启动后会显示 `Network URL: http://你的IP:8501`，局域网内其他设备直接访问该地址即可。

## 5.10.4 云服务器部署

```
1. 上传项目到服务器
scp -r "项目目录" user@server:/opt/

2. 服务器上安装
cd /opt/项目目录
pip install -r requirements.txt

3. 后台运行（使用 nohup 或 screen）
nohup python -m streamlit run uran/dashboard/research_app.py \
 --server.port 80 \
 --server.headless true \
 > streamlit.log 2>&1 &

4. 或使用 systemd 服务（推荐）
sudo vim /etc/systemd/system/uran-demo.service
```

**systemd 服务文件示例** ( `/etc/systemd/system/uran-demo.service` )：

```
[Unit]
Description=基于微内核的自演进 RAN 原型
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/opt/项目目录
ExecStart=/usr/bin/python3 -m streamlit run uran/dashboard/research_app.py \
 --server.port 8501 --server.headless true
Restart=always

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable uran-demo
sudo systemctl start uran-demo
```

5.10.5 依赖清单

| 库         | 版本要求    | 用途             |
|-----------|---------|----------------|
| streamlit | ≥1.30.0 | Web 仪表盘框架      |
| plotly    | ≥5.18.0 | KPI 曲线与拓扑图     |
| pandas    | ≥2.0.0  | 仿真数据 DataFrame |
| numpy     | ≥1.24.0 | 数值计算与随机数       |
| pydantic  | ≥2.0.0  | 数据模型与校验        |
| PyYAML    | ≥6.0.0  | 场景配置文件解析       |
| pytest    | ≥7.0.0  | 自动化测试          |
| kaleido   | ≥0.2.1  | 静态图表导出         |

第六部分：多阶段演进路线图

6.1 已达成阶段总览

模块1：科研仿真演示

验证链路自适应逻辑和可视化效果

✓ 已完成

↓

模块2：AI 编排与数字孪生验证

引入 AI Agent 生成候选方案，沙箱验证

✓ 已完成

↓

模块3/4：通信微内核与模块化

实现可加载模块、回退机制、运行时监控

✓ 已完成

↓

模块5：SDR 半实物验证

信号生成、噪声模拟、BER/EVM 测量

✓ 已完成 (Mock SDR)

↓

模块6：完整闭环自演进系统

数据回流、策略优化、多轮迭代

✓ 已完成

## 6.2 下一阶段展望（模块7+）

基于当前 模块1-6 的完整实现，后续阶段可沿以下方向推进：

| 方向             | 目标                   | 关键技术                                                    |
|----------------|----------------------|---------------------------------------------------------|
| 真实 LLM 接入      | 将规则基线 AI 替换为大语言模型    | GPT-4/Claude API、prompt engineering、function calling    |
| GNU Radio 硬件对接 | 使用 USRP 进行真实无线电收发    | <code>gnuradio_adapter.py</code> 实现、USRP B210 驱动、实时信号处理 |
| PHY 级精细仿真      | 3GPP 物理层级级别的精确仿真     | TDL/CDL 信道模型、PDSCH/PUSCH 链路级仿真                          |
| O-RAN 集成       | 对接 O-RAN RIC/xApp 框架 | RIC 适配器、xApp 桩实现、A1/E2 接口                               |
| 强化学习演进         | 从启发式优化升级到 RL 自演进     | PPO/SAC 算法、自定义 Gym 环境、在线策略更新                            |

## 第七部分：项目现状与产出清单

## 7.1 代码统计

| 层级                                                            | 文件数 | 说明                                           |
|---------------------------------------------------------------|-----|----------------------------------------------|
| AI 编排层 ( <code>uran/ai/</code> )                              | 11  | 5 Agent + 编排器 + 数据结构 + LLM适配器                |
| 数字孪生层 ( <code>uran/twin/</code> )                             | 6   | 沙箱 + 验证器 + 仿真器 + 故障注入 + 评分 + 报告              |
| 微内核运行时 ( <code>uran/runtime/</code> )                         | 6   | 内核 + 状态 + 事件 + 监控 + 安全 + 加载器                 |
| 通信模块库 ( <code>uran/modules/</code> )                          | 9   | 基类 + 注册 + 调制 + 编码 + HARQ + 导频 + 调度 + 功控 + 回退 |
| SDR 适配 ( <code>uran/sdr/</code> )                             | 5   | 基类 + Mock + 波形 + 频谱 + GNU Radio预留            |
| 演进闭环 ( <code>uran/evolution/</code> )                         | 6   | 经验 + 数据湖 + 策略记忆 + 优化器 + 推荐器 + 闭环             |
| 仿真核心 ( <code>uran/sim/</code> )                               | 9   | 节点/链路/拓扑/信道/自适应/KPI/记录/实验/加载                 |
| Dashboard ( <code>uran/dashboard/</code> )                    | 6   | 主应用 + 控制器 + 拓扑 + 图表 + 时间线 + 导出               |
| 端到端 ( <code>uran/demo/</code> )                               | 1   | 全流程集成                                        |
| 公共工具 ( <code>uran/common/</code> & <code>uran/utils/</code> ) | 9   | ID/时间/序列化/异常/默认参数/频段配置                       |
| 测试 ( <code>tests/</code> )                                    | 11  | 119 个测试用例                                    |
| 配置 ( <code>configs/</code> )                                  | 4   | 4 个 YAML 科研场景                                |
| 合计                                                            | ~80 | 完整的全阶段科研系统                                   |

## 7.2 文档产出

| 文档         | 文件                                            | 用途                    |
|------------|-----------------------------------------------|-----------------------|
| 项目思考与设计思路  | 设计思路.txt                                      | 原始设计理念、架构原则           |
| 初步设计文档集    | 初步设计.txt                                      | 架构设计/概要设计/框架设计/模块划分   |
| 模块1 详细设计   | 详细设计.txt                                      | 面向 AI IDE 的实现级设计说明    |
| 模块2-6 详细设计 | docs/设计文档/RAN-AI Phase II-V 科研演示系统完整详细设计文档.md | 多阶段完整技术规格             |
| 系统首页设计     | 首页设计.md                                       | 首页 UI 设计规范            |
| 概要设计文档     | docs/概要设计文档.md                                | 系统架构、模块划分             |
| 模块设计文档     | docs/模块设计文档.md                                | 各模块接口、数据模型            |
| 详细设计文档     | docs/详细设计文档.md                                | 类设计、方法签名              |
| 测试报告 V1    | docs/测试报告.md                                  | 模块1 17 个测试用例          |
| 测试报告 V2    | docs/AI_RAN_Demo_完整测试报告-V2.md                 | 全阶段 119 个测试用例详解       |
| 系统介绍 V1    | docs/系统介绍与用户手册.md                             | 模块1 立项汇报文档            |
| 本文档 V2     | docs/系统介绍与用户手册-V2.md                          | 模块1-6 全阶段立项汇报文档，含部署指南 |

## 7.3 测试覆盖

| 指标    | 数值                                      |
|-------|-----------------------------------------|
| 测试文件数 | 11                                      |
| 测试用例  | 119                                     |
| 通过率   | 100%                                    |
| 执行耗时  | < 2 秒                                   |
| 覆盖模块  | 全部 8 个子系统（AI/孪生/微内核/模块/SDR/演进/端到端/频段配置） |

---

## 附录 A: 关键技术术语表

---

| 术语    | 全称                                      | 解释                             |
|-------|-----------------------------------------|--------------------------------|
| gNB   | next Generation Node B                  | 5G 基站                          |
| UE    | User Equipment                          | 用户设备（手机等终端）                    |
| RAN   | Radio Access Network                    | 无线接入网                          |
| SNR   | Signal-to-Noise Ratio                   | 信噪比（dB），越大越好                   |
| SINR  | Signal-to-Interference-plus-Noise Ratio | 信干噪比（dB），扣除干扰后的实际信号            |
| BLER  | Block Error Rate                        | 块误码率（0~1），越小越好                 |
| BER   | Bit Error Rate                          | 比特误码率（0~1）                     |
| MCS   | Modulation and Coding Scheme            | 调制编码方案索引，越高越高效                 |
| HARQ  | Hybrid Automatic Repeat reQuest         | 混合自动重传请求                       |
| BPSK  | Binary Phase Shift Keying               | 二进制相移键控（最稳定）                   |
| QPSK  | Quadrature Phase Shift Keying           | 四进制相移键控                        |
| QAM   | Quadrature Amplitude Modulation         | 正交幅度调制<br>（16QAM/64QAM/256QAM） |
| IQ    | In-phase / Quadrature                   | 同相/正交分量，信号的复数表示                |
| EVM   | Error Vector Magnitude                  | 误差向量幅度（%），信号质量指标               |
| PSD   | Power Spectral Density                  | 功率谱密度，信号的频域"指纹"                |
| AWGN  | Additive White Gaussian Noise           | 加性白高斯噪声，最基本信道模型                |
| FFT   | Fast Fourier Transform                  | 快速傅里叶变换，用于频谱分析                 |
| SDR   | Software Defined Radio                  | 软件定义无线电                        |
| USRP  | Universal Software Radio Peripheral     | 通用软件无线电外设                      |
| ABI   | Application Binary Interface            | 应用二进制接口                        |
| Agent | —                                       | AI 智能体，自主执行特定任务的软件单元           |

## 附录 B: 项目资源

| 资源       | 路径/地址                                      |
|----------|--------------------------------------------|
| 项目根目录    | e:\AI RAN Demo\                            |
| 源代码      | e:\AI RAN Demo\uran\                       |
| 测试       | e:\AI RAN Demo\tests\                      |
| 场景配置     | e:\AI RAN Demo\configs\research_scenarios\ |
| Web 仪表盘  | http://localhost:8501                      |
| 设计文档     | 设计思路.txt , 初步设计.txt , 详细设计.txt             |
| 模块2-6 设计 | RAN-AI Phase II-V 科研演示系统完整详细设计文档.md        |
| 文档目录     | docs/                                      |

文档版本: V2.0

适用版本: Alpha 1.0 — 模块1-6 全阶段集成版, 含系统首页与频段配置

上一版本: V1.0 (模块1 演示版)

更新内容: 新增系统首页、频段配置 (含中国移动 n41/n79 方案)、部署指南; 模块1 控制栏重构

项目名称: 基于微内核的自演进 RAN 原型 (Microkernel-based Self-Evolving RAN Prototype)